

**Yenepoya Institute of Arts, Science,
Commerce, and Management
Final Project Report
on
Design and Development of Natural
Language Interface for Healthcare
Infrastructure**

Student Name: AYISHA NUHAILA

Industry Mentor: Mr. Shashank

Guided by: Mrs. MANJULA

Table of Contents

1. Executive Summary	3
2. Background	4
2.1 Aim	4
2.2 Technologies	5
2.3 Hardware Architecture	5
2.4 Software Architecture	6
3. System	8
3.1 Requirements	9
3.1.1 Functional Requirements	9
3.1.2 User Requirements	9
3.1.3 Environmental Requirements	10
3.2 Design and Architecture	10
3.3 Implementation	13
3.4 Testing	17
3.5 Graphical User Interface (GUI) Layout	19
3.6 Customer Testing	21
3.7 Evaluation.....	22
4. Snapshots of the Project	24
5. Conclusions	30
6. Further Development or Research	31
7. References	32
8. Appendix	33
8.1 Weekly Progress Reports	34

1. Executive Summary

The AI-Based Healthcare Chatbot System is a web-based application designed to provide basic healthcare assistance through an interactive conversational interface. It enables users to input symptoms and receive instant responses, helping them understand common health conditions and possible precautions.

The system is developed using Python with the Flask framework for backend processing, while the frontend is built using HTML, CSS, and JavaScript. It follows a client-server architecture where user inputs are sent to the server, processed using chatbot logic, and returned as meaningful responses.

The chatbot operates on a rule-based mechanism, where predefined keywords are matched with user input to generate appropriate replies. It can handle common symptoms such as fever, cold, and headache, providing general advice like rest, hydration, and medical consultation when necessary.

The system is lightweight, easy to use, and provides quick responses, making it suitable for basic healthcare guidance. Testing shows that it performs efficiently for simple queries, though it is limited by its predefined knowledge base and cannot provide advanced medical diagnosis.

In conclusion, this project demonstrates the effective use of chatbot technology in healthcare applications and highlights its potential for future enhancement using advanced artificial intelligence and natural language processing techniques.

2. Background

The rapid growth of digital technology has significantly transformed the healthcare sector, making services more accessible and efficient. One such advancement is the use of chatbot systems, which are designed to simulate human conversation and provide automated responses to user queries. These systems are increasingly being used in healthcare to assist patients with basic information, symptom checking, and general guidance.

Traditionally, individuals had to rely on direct consultation with healthcare professionals even for minor health concerns. This often leads to overcrowding in hospitals, increased waiting times, and unnecessary pressure on medical staff. In many cases, people may delay seeking advice due to inconvenience, cost, or lack of access to medical facilities. As a result, there is a growing need for systems that can provide immediate and basic healthcare support.

Healthcare chatbots address this gap by offering instant responses and continuous availability. They can guide users by analyzing symptoms and suggesting possible actions, such as rest, hydration, or consulting a doctor. While advanced chatbot systems use artificial intelligence and natural language processing, simpler rule-based systems are still effective for handling common queries.

This project focuses on developing a rule-based healthcare chatbot system that provides quick and user-friendly assistance. By integrating web technologies with backend processing, the system demonstrates how digital tools can support healthcare services and improve user accessibility.

2.1 Aim

The aim of this project is to develop a healthcare chatbot that provides basic medical guidance by analyzing user symptoms and generating appropriate responses using a rule-based approach.

- To design and develop an AI-based healthcare chatbot system
- To provide basic medical guidance for common health issues
- To enable users to interact through a simple and user-friendly interface
- To process user input and generate appropriate responses
- To integrate frontend and backend components effectively

- To implement chatbot logic using a rule-based approach
- To develop the system using Python and Flask

2.2 Technologies

The system is developed using Python and Flask for backend processing, while HTML, CSS, and JavaScript are used to create a responsive and user-friendly frontend interface. These technologies work together to enable smooth communication between the user and the chatbot, ensuring efficient input processing and response generation.

- Providing basic health guidance
- Responding to symptoms like fever, cold, headache
- Helping users with initial medical advice
- Here are 2 extra healthcare-focused points you can add:
 - To improve accessibility to basic healthcare information for users anytime and anywhere
 - To reduce the workload on healthcare professionals by handling common and repetitive queries

2.3 Hardware Architecture

The hardware architecture of the Healthcare Chatbot System represents the physical components required for the proper functioning of the application. Since this is a web-based software project, the hardware requirements are simple and user-friendly.

- Client System / User Device – Desktop, laptop, or mobile device used to access the chatbot
- Processor – Intel i3 / i5 or equivalent for smooth execution
- RAM – Minimum 4 GB recommended
- Storage – At least 500 MB free space for project files and dependencies

- Input Devices – Keyboard, mouse, or touchscreen for user interaction
- Network Connection – Internet or local network for accessing the web application
- Server System – System hosting the Flask application and backend logi

These hardware components work together to ensure the proper functioning of the healthcare chatbot application.

2.4 Software Architecture

The software architecture of the healthcare chatbot system is designed using a client-server model, where the application is divided into frontend and backend components that work together to process user requests and generate responses.

1. Presentation Layer (Frontend)

- Handles user interaction through a web-based interface
- Developed using HTML, CSS, and JavaScript
- Provides forms and dashboards for patients to interact with the system
- Includes a chatbot interface for natural language communication

2. Application/Business Layer

- Implemented using Python and Flask
- Processes user requests such as booking appointments and retrieving doctor information
- Handles chatbot logic to interpret and respond to user queries
- Validates input data (e.g., appointment time, user details)
- Manages communication between frontend and data layer using REST APIs

3. Persistence Layer

- Stores healthcare-related data such as:
 - Patient details
 - Doctor information
 - Appointment records
- Uses JSON file or database for data storage
- Handles data retrieval, updates, and storage operations
- Ensures data consistency during booking and updates

Architectural characteristics:

- Follows client-server architecture
- Uses RESTful API communication between frontend and backend
- Ensures separation of concerns between UI, logic, and data
- Provides real-time response to user actions
- Supports easy future upgrades (e.g., database integration, AI chatbot)

3. System

The proposed system is a web-based healthcare application designed to improve accessibility and interaction within healthcare infrastructure using a natural language interface. The system enables patients to communicate with healthcare services through simple text-based queries, eliminating the need for complex navigation or technical knowledge. Users can perform essential operations such as viewing doctor details, checking availability, booking appointments, and receiving assistance through an integrated chatbot. The chatbot plays a key role in the system by understanding user input and providing relevant responses related to healthcare services, appointment status, and general information.

Technically, the system follows a client-server architecture where the frontend is developed using HTML, CSS, and JavaScript to provide an interactive and user-friendly interface, while the backend is implemented using Python and Flask to handle business logic and request processing. Communication between the frontend and backend is carried out through RESTful API calls, ensuring smooth data exchange and real-time updates. The system uses a structured storage mechanism such as JSON or a database to maintain patient records, doctor details, and appointment data.

When a user interacts with the system, the request is captured by the frontend and sent to the backend server. The backend validates the input, processes the required operation, and updates or retrieves data from the storage layer. The processed response is then sent back to the frontend, which dynamically updates the interface to reflect the latest information. This ensures consistency, accuracy, and immediate feedback to the user. Overall, the system demonstrates a modular and efficient design that enhances usability and provides a strong foundation for future improvements such as AI-based diagnosis, real-time consultation, and advanced natural language processing capabilities.

3.1 Requirements

The requirements of this system are defined to ensure that the healthcare application is functional, user-friendly, and capable of supporting natural language interaction. These requirements are divided into functional, user, and environmental categories.

3.1.1 Functional Requirements

The system must provide the following functionalities:

- Display available doctors along with their specialization, timings, and details
- Allow users to book appointments with doctors
- Validate appointment inputs such as date, time, and availability
- Store and manage patient information and appointment records
- Provide a chatbot interface for natural language interaction
- Process user queries such as “book appointment,” “show doctors,” or “check status”
- Display appointment confirmation and status to the user
- Handle invalid inputs and return appropriate error messages
- Update and retrieve data dynamically from the storage system

3.1.2 User requirements

The system should meet the following user expectations:

- Simple and easy-to-use interface for all users
- Minimal steps required to book an appointment
- Clear display of doctor and appointment information
- Quick response to user actions and queries
- Assistance through chatbot for guidance and support

- Accurate and reliable system behavior
- Easy navigation between different sections of the system

3.1.3 Environmental requirements

The system should operate in the following environment:

- Operating System: Windows, Linux, or macOS
- Backend: Python 3.x with Flask framework
- Frontend: HTML, CSS, JavaScript (modern browser support)
- Browser: Chrome, Edge, Firefox, or any updated browser
- Storage: JSON file or database for storing healthcare data
- Network: Not required for local system, but needed for deployment
- System Access: Localhost (for development) or web server (for deployment)

3.2 Design and Architecture

The design follows a client-server model with unidirectional request-response flow:

- Centralized backend for handling patient queries and chatbot logic
- No direct frontend access to sensitive healthcare data
- System updates patient data and responses after each request
- Chatbot uses keyword/action-based routing for accurate replies
- Clear APIs for future mobile or healthcare system integration
- JSON storage for patient data and chatbot knowledge

Key design decisions:

- Centralized backend for handling patient queries and chatbot logic
- No direct frontend access to sensitive healthcare data
- System updates responses after each user interaction
- Chatbot uses keyword/action-based routing for accurate replies
- Clear API design for future healthcare system integration

Purpose: Transfers server-side product/cart/overview data to frontend JavaScript state.

```
fetch("/chat", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({ message: userInput })  
})
```

Observed result:

UI starts with real data immediately and remains synchronized after dynamic updates.

Purpose: Computes cart item count and total amount from current data state.

```
def run_cli():
    print("=" * 60)
    print(f" {PROJECT_NAME}")
    print(" Natural Language Interface for Healthcare")
    print("=" * 60)
    print("\nHello! Type a healthcare question and press Enter.")
    print("Type 'help' to see all commands.")
    print("Type 'exit' or 'quit' to stop.\n")
```

Observed result: Total amount displayed in cart matches product price × quantity values.

Purpose: Sends products, cart, and overview in one API response for efficient frontend refresh.

```
def run_cli():
    print("=" * 60)
    print(f" {PROJECT_NAME}")
    print(" Natural Language Interface for Healthcare")
    print("=" * 60)
    print("\nHello! Type a healthcare question and press Enter.")
    print("Type 'help' to see all commands.")
    print("Type 'exit' or 'quit' to stop.\n")

    # show some sample commands at startup
    print("Sample commands you can try:")
    for cmd in COMMANDS[:8]:
        print("  ", cmd["prompt"])
    print("  ... and more! Type 'help' for full list.\n")

    while True:
        user_input = input("You: ").strip()
```

Observed result: Frontend updates all dashboard blocks after add/checkout without full page reload.

Security-aware design choices:

- Input type validation for quantity
- Required field checks in POST body
- Error handling for missing or malformed requests
- Escaped rendering on frontend to reduce injection risk in dynamic content

3.3 Implementation

Backend implementation in app.py includes:

- `load()` and `save(data)` for JSON persistence.
- `build_cart_summary(db)` for item count and total.
- `build_overview(db)` for dashboard counters.
- `find_product(products, query)` for chatbot product matching.
- `build_chat_response(action, message)` for assistant responses.

Purpose: Returns all values required to refresh dashboard sections after user actions.

```
# api to get command buttons for the website
if path == "/api/commands":
    data = {
        "project": PROJECT_NAME,
        "welcome": WELCOME_MESSAGE,
        "commands": get_quick_commands()
    }
    self.send_json(data)
    return

# 404 if route not found
self.send_json({"error": "not found"}, code=404)
```

Observed result: Cart and stock update correctly, and invalid requests return proper error messages.

Core endpoints:

- GET / renders dashboard template.
- GET /dashboard-data returns complete refresh payload.
- GET /products returns product list.
- GET /cart returns cart list.
- POST /add validates and updates cart/stock.
- GET /checkout places order and clears cart.
- POST /chatbot responds to action/message input.

Purpose: Clears cart on successful order placement and returns updated summaries.

```
@app.route("/checkout")
def checkout():
    db = load()

    if not db["cart"]:
        return jsonify({"msg": "cart is empty"}), 400

    db["cart"] = []
    save(db)
    return jsonify(
        {
            "msg": "order placed",
            "cart_summary": build_cart_summary(db),
            "overview": build_overview(db),
        }
    )
```

Observed result: Prevents checkout on empty cart and completes checkout for valid cart.

Frontend implementation in static/app.js includes:

- State object to mirror server data.
- refreshDashboard() to sync UI after operations.
- addToCart(productName) to invoke add route.
- checkout() for order completion.
- renderCart() for cart panel rendering.
- updateProductStocks(products) for live stock updates.
- Chat functions for quick action chips and typed messages.

Purpose: Sends selected product and quantity to backend and refreshes UI after success.

```
async function addToCart(productName) {
  const select = document.querySelector(`[data-qty-select="${cssEscape(productName)}"]`);
  const qty = Number(select ? select.value : 1);

  const response = await fetch("/add", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ product: productName, qty }),
  });

  const result = await response.json();

  if (!response.ok) {
    setStatus(result.msg || "Unable to add product.", true);
    return;
  }

  setStatus(`${productName} added to cart successfully.`);
  await refreshDashboard();
}
```

Observed result: User receives immediate status message and refreshed cart/stock values.

Template implementation in templates/index.html includes:

- Product cards with quantity selector.
- Cart summary panel and checkout button.
- Customer profile section.
- Assistant section with quick actions and input.
- Initial server data injection into window. `__INITIAL_DATA__`

3.4 Testing

Testing was performed through systematic manual scenarios and API validation checks.

Functional test scenarios:

- Product list loads on page open.
- Add-to-cart with valid quantity succeeds.
- Add-to-cart with quantity beyond stock fails with message.
- Non-numeric or non-positive quantity returns error.
- Cart overview updates after each add.
- Checkout succeeds only when cart has items.
- Empty-cart checkout returns expected error.
- Chatbot quick actions return structured helpful responses.
- Typed queries like “show cart”, “payment options”, product names return relevant output.

```
@app.route("/checkout")
def checkout():
    db = load()

    if not db["cart"]:
        return jsonify({"msg": "cart is empty"}), 400

    db["cart"] = []
    save(db)
    return jsonify(
        {
            "msg": "order placed",
            "cart_summary": build_cart_summary(db),
            "overview": build_overview(db),
        }
    )
```

Data consistency tests:

- Stock decreases when item is added to cart.
- Cart quantity increments for repeated product additions.
- Checkout clears cart entries.
- Overview counters match current data after refresh.

Usability tests:

- Buttons and sections are discoverable.
- Status messages are visible and understandable.
- Navigation buttons scroll to intended sections.

- Chat stream remains readable during multiple interactions.

Known testing limitations:

- No automated test suite in current version.
- No high-concurrency simulation due to file-based storage.
- No cross-browser automation scripts included.

3.5 Graphical User Interface (GUI) Layout

The GUI is organized as a dashboard with clear visual hierarchy:

- **Top Navigation:** quick jump links to products, inventory stats, and cart.
- **Hero Section:** introduction and primary action buttons.
- **Product Section:** cards showing product details, stock badges, delivery information, and quantity selectors.
- **Overview Metrics:** product count, available units, cart items, and cart total.
- **Cart Section:** selected items, quantities, total, and checkout action.
- **Customer Section:** static user profile snapshot.
- **Assistant Section:** chatbot stream, quick action chips, and text input.

UI principles followed:

- Consistent spacing and card-based grouping.
- Action buttons near relevant content.
- Immediate feedback messages.
- Readable typography and contrast.

- Minimal interaction friction.

Purpose: Renders cart rows dynamically with product name, quantity, and price.

```
function renderCart() {  
  if (!cartList) {  
    return;  
  }  
  if (!state.cart.length) {  
    cartList.innerHTML = `

Cart is empty</p>`;  
    return;  
  }  
  
  cartList.innerHTML = state.cart  
    .map(  
      (item) => {  
        const product = state.products.find((entry) => entry.name === item.product);  
        const price = product ? product.price : 0;  
        return `  
          <article class="cart-item">  
            <strong>${escapeHtml(item.product)}</strong>  
            <span>Quantity: ${escapeHtml(item.qty)}</span>  
            <span>Price: ${currency(price)}</span>  
          </article>  
        `;  
      }  
    )  
    .join("");  
}


```

Observed result: Cart panel always reflects latest items and prices after every operation.

Purpose: Defines cart panel layout and server-rendered fallback content

```
<section class="panel" id="cart-section">
  <div class="panel-heading">
    <div>
      <h2>Cart</h2>
    </div>
    <span class="chip" id="cart-count-chip">{{ overview.cart_items }} items</span>
  </div>

  <div id="cart-list" class="cart-list">
    {% if cart %}
      {% for item in cart %}
        {% set matched = (products | selectattr("name", "equalto", item.product) | list | first) %}
        <article class="cart-item">
          <strong>{{ item.product }}</strong>
          <span>Quantity: {{ item.qty }}</span>
          <span>Price: Rs. {{ matched.price if matched else 0 }}</span>
        </article>
      {% endfor %}
    {% else %}
      <p class="empty-copy">Cart is empty</p>
    {% endif %}
  </div>

  <div class="total-row">
    <span>Total</span>
    <strong id="cart-total-text">Rs. {{ overview.cart_total }}</strong>
  </div>
```

Observed result: Initial page load already shows consistent cart data without waiting for JS calls.

3.6 Customer testing

A small user feedback exercise was conducted with peer users to evaluate practicality.

Observed outcomes:

- Users quickly understood product browsing and add-to-cart behavior.
- Quick action chatbot buttons improved discoverability of support features.
- Cart summary helped users track progress before checkout.
- Most users completed a mock order without external assistance.

Common feedback:

- Add remove/decrement buttons in cart.
- Add user login and personalized order history.
- Show order confirmation details page after checkout.
- Include category filters and search in product grid.

Interpretation:

- Core workflow is intuitive and stable.
- UX is strong for a first full-stack release.
- Enhancement opportunities mainly relate to scale and advanced features.

3.7 Evaluation

The project successfully meets its core objectives:

- End-to-end e-commerce workflow implemented.
- Website migration completed from original CLI concept.
- Frontend-backend integration demonstrated clearly.
- Persistent state management works reliably for demo scope.
- Chatbot assistance increases interaction quality.

Strengths:

- Clear modular architecture.
- Straightforward API design.
- Practical real-world flow representation.

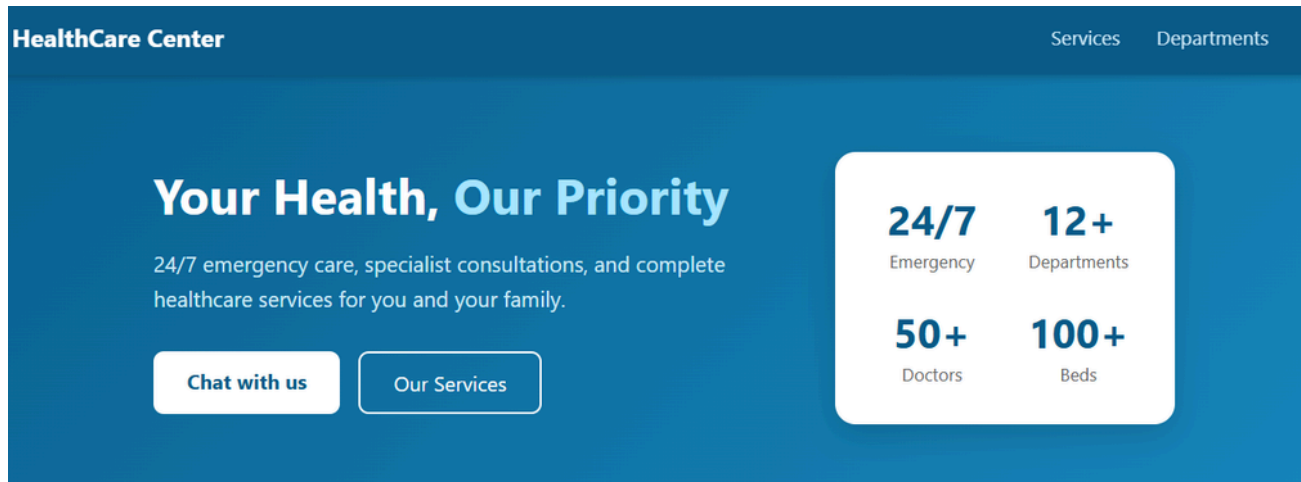
- Easy to explain during viva and interviews.

Limitations:

- JSON file storage limits scalability and concurrent safety.
- No authentication/authorization.
- Limited production-grade security controls.
- Manual testing dominant; automation minimal.

4. Snapshots

i) Home Landing View



ii) Product card

Our Services



Emergency Care

24/7 emergency response at Gate 1. Call +91 1800 123 CARE for immediate help.



Lab & Diagnostics

Blood tests, ECG, X-ray, CT & MRI referrals all available on campus.



Pharmacy

Main pharmacy open 7AM–10PM. Emergency counter available round the clock.



OPD Consultations

General and specialist doctors available from 9AM to 6PM on working days.



ICU & Critical Care

Fully operational ICU with trained staff and monitoring equipment.



Online Consultation

Book a video consultation 9AM–8PM. No need to travel for basic queries.

iii) Cart



iv) Checkout

Contact Us

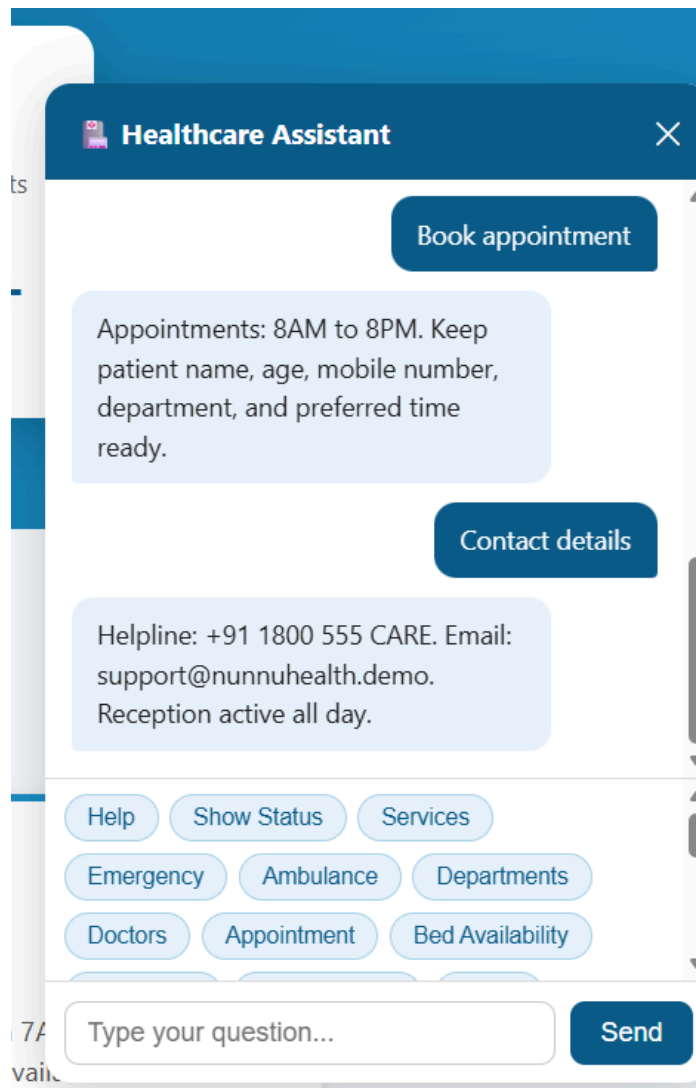
**Helpline**
+91 1800 555 CARE

**Email**
support@nunuhealth.demo

**Address**
Wellness Avenue,
Sector 7, near City Park

**Visiting Hours**
11AM–1PM and 5PM–
7PM

v) Chatbot Quick response



vi) Chatbot Typed Query Response

Current status: Emergency is active 24/7, OPD is open 8AM to 8PM, 12 general beds available, 3 ICU beds available, pharmacy is open.

[Help](#)[Show Status](#)[Services](#)[Emergency](#)[Ambulance](#)[Departments](#)[Doctors](#)[Appointment](#)[Bed Availability](#)

5. Conclusions

This project demonstrates the successful development of a web-based **natural language interface for healthcare infrastructure**. It validates full-stack concepts by integrating a structured frontend, REST-style backend, and persistent data layer into one coherent system.

The implemented architecture supports practical healthcare functionalities such as patient query handling, symptom understanding, and chatbot-based guidance. The system also emphasizes maintainability through modular code organization and clear separation of responsibilities.

From an academic perspective, the project satisfies objectives related to software design, implementation, testing, user interface development, and system evaluation. It also provides a strong foundation for future enhancements such as advanced medical diagnosis support, integration with hospital systems, and telemedicine services.

6. Further development or research

Recommended future scope:

- Integrate relational database (MySQL/PostgreSQL) for secure and reliable patient data management
- Add authentication and role-based access for patients, doctors, and administrators
- Support patient history tracking and medical records management
- Improve chatbot with advanced NLP models and contextual understanding
- Enable appointment booking and scheduling features
- Integrate with hospital management systems and electronic health records (EHR)
- Implement real-time health monitoring and alerts
- Add multilingual support for better patient communication
- Introduce automated testing (unit, API, UI) for system reliability
- Deploy system on cloud with CI/CD for scalability and availability
- Add analytics dashboard for patient data insights and healthcare trends
- Apply caching strategies to improve system performance at scale

Research directions:

- Enhancing chatbot accuracy using advanced NLP and machine learning models
- Improving context-aware conversations for better patient interaction
- Integration with electronic health records (EHR) for real-time data access
- Developing multilingual language interfaces for wider accessibility

7. References

- Flask – <https://flask.palletsprojects.com/en/stable/>
- JavaScript (MDN Docs) – <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- HTML – <https://developer.mozilla.org/en-US/docs/Web/HTML>
- CSS – <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Fetch API – https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- JSON – <https://www.json.org/json-en.html>
- Natural Language Processing – <https://www.ibm.com/topics/natural-language-processing>
- World Health Organization – <https://www.who.int/>
- Python Documentation – <https://docs.python.org/3/>
- Bootstrap Documentation – <https://getbootstrap.com/>

8. Appendix

- Complete source file list:
 - app.py
 - templates/index.html
 - static/app.js
 - static/styles.css
 - data.json (patient data / chatbot knowledge base)
 - main.py (optional CLI version)
- API request/response samples:
 - POST /chat – patient message input and chatbot response
 - GET /health-data – sample patient data response
 - POST /process-query – symptom input and system output
- Additional Appendix Details
 - Test case checklist and outcomes – verifies chatbot accuracy, input handling, and system responses
 - Setup and run instructions – steps to install dependencies and run the Flask application
 - Known issues and workaround notes – limitations and temporary fixes in the system
 - Additional UI snapshots – screenshots of chatbot interface and user interactions

8.1 Weekly report

10-03-2020

WEEKLY PROJECT PROGRESS REPORT (WPPR)–

For week commencing 7 March 2026

Programme: BCA (Artificial intelligence, Cloud Computing & Devops)

Student Name: **AYISHA NUHAILA**

WPPR: **WEEK 1**

Register Number: **238CAICD031**

Internal Guide's Name: **MANJULA**

MAJOR PROJECT TITLE:

Design and Development of Natural Language Interface for Healthcare Infrastructure

TARGETS SET FOR THE WEEK:

Targets Set for the Week: This week, the primary targets were to finalize the project title and define the problem statement clearly, understand the scope of building a Chatbot CLI for Healthcare using Python and Basic String Matching, and review existing literature on NLP-based chatbots in the healthcare domain. The team also aimed to set up the development environment including VS Code and Python and prepare an initial project plan covering all deliverables.

Progress/Achievements for the week:

The project title was successfully finalized as "Design and Development of Natural Language Interface for Healthcare Infrastructure." A preliminary understanding of Basic String Matching and if/else logic-based chatbot development was achieved. VS Code and Python were installed and configured successfully. An initial list of healthcare-related queries and pre-defined responses was drafted to guide the chatbot logic.

10-03-2020

Future Work Plans:

In the coming week, the focus will be on designing the chatbot architecture and planning the command-loop structure for CLI interaction. Work will begin on mapping healthcare keywords to pre-defined responses and drafting the Project Proposal document as the first major deliverable.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

- Weekly project report for week commencing- 09 March

25-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)–**For week commencing 7 March 2026****Programme: BCA (Artificial intelligence, Cloud Computing & Devops)**Student Name: **AYISHA NUHAILA**WPPR: **WEEK 2**Internal Guide's Name: **MANJULA**Register Number: **23BCAICD031****MAJOR PROJECT TITLE:****Design and Development of Natural Language Interface for Healthcare Infrastructure****TARGETS SET FOR THE WEEK:**

This week, the targets were to begin drafting the Project Proposal document, design the basic chatbot architecture, and identify all the healthcare-related keywords and queries the chatbot should handle. The team also aimed to plan the command-loop structure and define the input-output flow of the CLI tool.

Progress/Achievements for the week:

The Project Proposal document was drafted and reviewed with an outline of objectives, scope, and methodology. A basic chatbot architecture was designed showing how user input flows through the string matching logic to produce responses. A list of healthcare keywords such as "Show status," "Help," "Appointment," and "Emergency" was prepared and mapped to pre-defined responses.

25-03-2026

Future Work Plans:

Next week will focus on beginning the actual Python coding of the chatbot CLI, implementing the basic command loop, and writing the first set of if/else string matching conditions. The Low Level Design (LLD) document will also be started.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

25-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)–**For week commencing 7 March 2026****Programme: BCA (Artificial intelligence, Cloud Computing & Devops)**Student Name: **AYISHA NUHAILA**WPPR: **WEEK 3**Internal Guide's Name: **MANJULA**Register Number: **23BCAICD031****MAJOR PROJECT TITLE:****Design and Development of Natural Language Interface for Healthcare Infrastructure****TARGETS SET FOR THE WEEK:**

The targets for the week were to design the overall chatbot architecture and define the structure of the command-line interface (CLI) interaction. This included planning how user inputs would be processed, how keywords would be identified, and how responses would be generated using basic string matching techniques. The aim was to create a clear blueprint for the chatbot system.

Further targets included preparing flowcharts and logic design for the chatbot's working process and finalizing the structure of the dataset containing healthcare-related queries and responses. The plan also involved deciding how different conditions would be handled using if/else logic to ensure smooth interaction.

Progress/Achievements for the week:

During this week, the chatbot architecture was successfully designed, outlining the flow from user input to response generation. The structure of the CLI interaction was defined, and the logic for handling user queries using string matching and conditional statements was developed. Flowcharts and basic logic diagrams were also created to represent the working of the system.

In addition, the dataset of healthcare-related queries and responses was refined and organized properly. The mapping between keywords and responses was established, which helped in improving the clarity and efficiency of the chatbot design. This provided a strong base for starting the implementation phase.

25-03-2026

Future Work Plans:

In the upcoming week, the focus will be on implementing the chatbot using Python and developing the command-loop for continuous user interaction. Initial testing will be carried out with basic queries to evaluate system performance. Further improvements will also be planned to enhance response accuracy and handling of different user inputs.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

08-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)–**For week commencing 7 March 2026****Programme: BCA (Artificial intelligence, Cloud Computing & Devops)**Student Name: **AYISHA NUHAILA**WPPR: **WEEK 4**Internal Guide's Name: **MANJULA**Register Number: **23BCAICD031****MAJOR PROJECT TITLE:****Design and Development of Natural Language Interface for Healthcare Infrastructure****TARGETS SET FOR THE WEEK:**

The targets for the week were to begin implementing the chatbot using Python and develop the command-line interface (CLI) interaction loop. This included writing code to process user input, apply basic string matching, and generate responses using if/else conditions. The aim was to transform the designed system into a working prototype. Further targets included integrating the prepared dataset of healthcare queries and responses into the program. The plan also involved setting up initial testing to check how effectively the chatbot responds to user inputs and identifying any issues in the logic or flow.

Progress/Achievements for the week:

During this week, the chatbot implementation was successfully initiated using Python, and a command-loop was created for continuous user interaction. The system was able to accept user queries and respond using predefined conditions based on keyword matching, marking the development of a functional prototype. In addition, the dataset was integrated into the chatbot, and initial testing was performed with basic healthcare queries. Some minor issues were identified and corrected, which improved the response handling. The chatbot demonstrated the ability to handle simple queries effectively.

08-04-2026

Future Work Plans:

In the upcoming week, the focus will be on improving the chatbot's response accuracy and expanding its ability to handle more complex healthcare queries. Additional keywords and responses will be added, and improvements will be made to the matching logic. Further testing and debugging will also be carried out to enhance system performance and reliability.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

15-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)–**For week commencing 7 March 2026****Programme: BCA (Artificial intelligence, Cloud Computing & Devops)**Student Name: **AYISHA NUHAILA**WPPR: **WEEK 5**Internal Guide's Name: **MANJULA**Register Number: **23BCAICD031****MAJOR PROJECT TITLE:****Design and Development of Natural Language Interface for Healthcare Infrastructure****TARGETS SET FOR THE WEEK:**

The targets for the week were to improve the chatbot's response accuracy and enhance its ability to handle a wider range of healthcare-related queries. This included expanding the dataset with more keywords and predefined responses, and refining the string matching logic to better interpret user inputs. The aim was to make the chatbot more effective and reliable.

Further targets included strengthening the integration between different parts of the system and ensuring smooth execution of the chatbot program. The plan also involved preparing the system to handle slightly complex queries and improving the overall flow of interaction.

Progress/Achievements for the week:

During this week, the chatbot's performance was improved by adding more healthcare-related queries and responses to the dataset. The string matching logic was refined, allowing the system to better understand user inputs and provide more accurate responses. This resulted in a noticeable improvement in the chatbot's functionality.

In addition, the system was tested with different types of inputs, and adjustments were made to enhance response handling. The integration of components became more stable, and the chatbot was able to handle a broader set of queries effectively. Overall performance and reliability were significantly improved.

15-04-2026

Future Work Plans:

In the upcoming week, the focus will be on conducting detailed system testing and debugging to identify and fix any remaining issues. Performance optimization will be carried out to improve efficiency and response time. Further improvements will also be made to handle complex queries more effectively and ensure system stability.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

22-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)–**For week commencing 7 March 2026****Programme: BCA (Artificial intelligence, Cloud Computing & Devops)**Student Name: **AYISHA NUHAILA**WPPR: **WEEK 6**Internal Guide's Name: **MANJULA**Register Number: **23BCAICD031****MAJOR PROJECT TITLE:****Design and Development of Natural Language Interface for Healthcare Infrastructure****TARGETS SET FOR THE WEEK:**

The targets for the week were to perform detailed system testing and debugging of the chatbot to ensure it works reliably under different conditions. This included identifying errors in response handling, improving the accuracy of keyword matching, and optimizing the overall performance of the system. The aim was to refine the chatbot into a stable and efficient application.

Further targets included enhancing the chatbot's ability to handle more complex healthcare queries and improving the interaction flow in the CLI. The plan also involved making minor improvements to code structure for better readability and maintainability.

Progress/Achievements for the week:

During this week, extensive testing was carried out using various healthcare-related queries, which helped in identifying and fixing multiple issues in the chatbot. The performance of the system improved after refining the matching logic and optimizing response generation. The chatbot became more stable and consistent in handling user inputs.

In addition, improvements were made to handle slightly complex queries more effectively, and the interaction flow was made smoother. Most of the major issues were resolved, and the system reached a near-final stage of development with reliable performance.

22-04-2026

Future Work Plans:

In the upcoming week, the focus will be on finalizing the system by completing integration and making final improvements to the chatbot interface. Documentation and report preparation will be completed, along with final testing to ensure the system is ready for demonstration and submission.

Implementation shown: No

☐

Yes

☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide